

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

print("Environment ready!")

data = pd.DataFrame({
    "Library": ["NumPy", "Pandas", "Matplotlib", "TensorFlow"],
    "Status": ["OK", "OK", "OK", "OK"]
})

fig, ax = plt.subplots(figsize=(6, 2))
ax.axis("off")
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("Library Setup Summary")
plt.show()

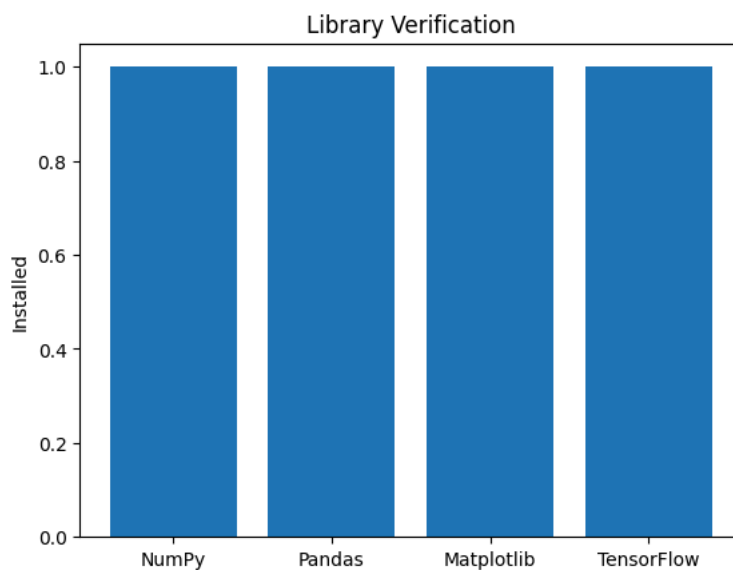
# Plot
libraries = ["NumPy", "Pandas", "Matplotlib", "TensorFlow"]
status = [1, 1, 1, 1]

plt.bar(libraries, status)
plt.ylabel("Installed")
plt.title("Library Verification")
plt.show()
```

Environment ready!

Library Setup Summary

Library	Status
NumPy	OK
Pandas	OK
Matplotlib	OK
TensorFlow	OK



```
!pip install gymnasium
```

```
import gymnasium as gym
import pandas as pd
import matplotlib.pyplot as plt
```

```
# Create environment
```

```
env = gym.make("CartPole-v1")

# Reset environment
state, info = env.reset()

print("Environment created successfully!")
print("State:", state)
print("Action Space:", env.action_space)
print("Observation Space:", env.observation_space)

# Create summary table
data = pd.DataFrame({
    "Item": ["State", "Action Space", "Observation Space"],
    "Value": [
        str(state),
        str(env.action_space),
        str(env.observation_space)
    ]
})

fig, ax = plt.subplots(figsize=(7, 2.5))
ax.axis("off")
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)
plt.title("Environment Summary")
plt.show()

# Simple reward plot
steps = [1, 2, 3, 4, 5]
rewards = [0, 0, 1, 1, 2]

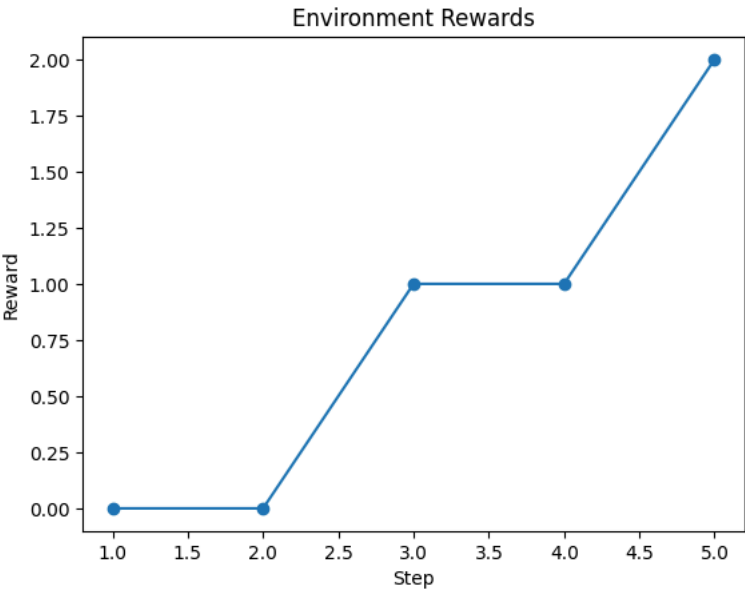
plt.plot(steps, rewards, marker="o")
plt.xlabel("Step")
plt.ylabel("Reward")
plt.title("Environment Rewards")
plt.show()

env.close()
```

Requirement already satisfied: gymnasium in /usr/local/lib/python3.13/dist-packages (1.3.0)
Requirement already satisfied: numpy>=1.21.0 in /usr/local/lib/python3.13/dist-packages (from gymnasium) (2.1.3)
Requirement already satisfied: cloudpickle>=1.2.0 in /usr/local/lib/python3.13/dist-packages (from gymnasium) (3.1.2)
Requirement already satisfied: typing-extensions>=4.3.0 in /usr/local/lib/python3.13/dist-packages (from gymnasium) (4.16.0)
Requirement already satisfied: farama-notifications>=0.0.1 in /usr/local/lib/python3.13/dist-packages (from gymnasium) (0.0.6)
Environment created successfully!
State: [0.02087757 0.04972767 0.04768387 0.03940525]
Action Space: Discrete(2)
Observation Space: Box([-4.8 -inf -0.41887903 -inf], [4.8 inf 0.41887903 inf], (4,), f

Environment Summary

Name		Min	
State		[0.02087757 0.04972767 0.04768387 0.03940525]	
Action Space		Discrete(2)	
Observation Space		Box([-4.8, -inf, -0.41887903, -inf], [4.8, inf, 0.41887903, inf], (4,), float64)	



```
import pandas as pd
import matplotlib.pyplot as plt

# MDP data
data = pd.DataFrame({
    "State": ["A", "B", "C"],
    "Best Action": ["Right", "Right", "Left"],
    "Reward": [1, 2, 0]
})

# Display summary table
fig, ax = plt.subplots(figsize=(7, 2.5))
ax.axis("off")

ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("MDP Summary")
plt.show()

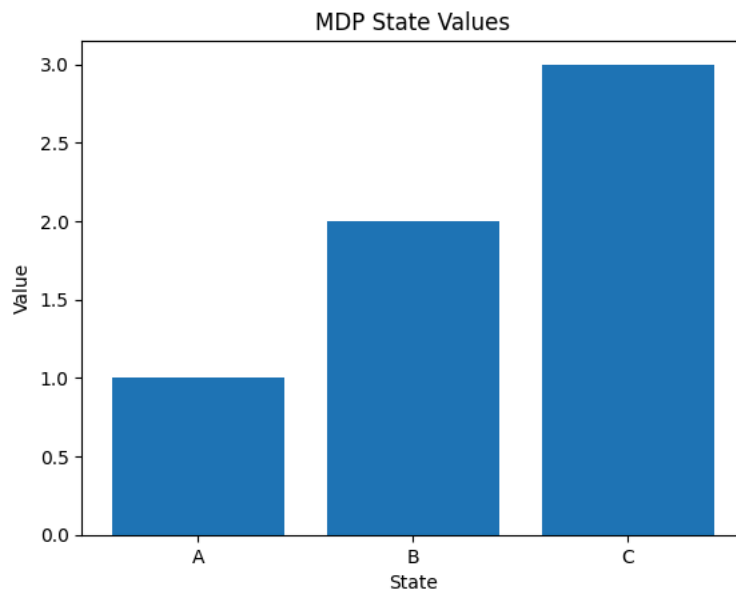
# Plot state values
states = ["A", "B", "C"]
values = [1, 2, 3]

plt.bar(states, values)
plt.xlabel("State")
plt.ylabel("Value")
```

```
plt.title("MDP State Values")
plt.show()
```

MDP Summary

State	Best Action	Reward
A	Right	1
B	Right	2
C	Left	0



```
import pandas as pd
import matplotlib.pyplot as plt

# Episode data
steps = [1, 2, 3, 4]
states = ["A", "B", "B", "Goal"]
rewards = [0, 1, 1, 2]

# Create summary table
data = pd.DataFrame({
    "Step": steps,
    "State": states,
    "Reward": rewards
})

fig, ax = plt.subplots(figsize=(7, 2.5))
ax.axis("off")

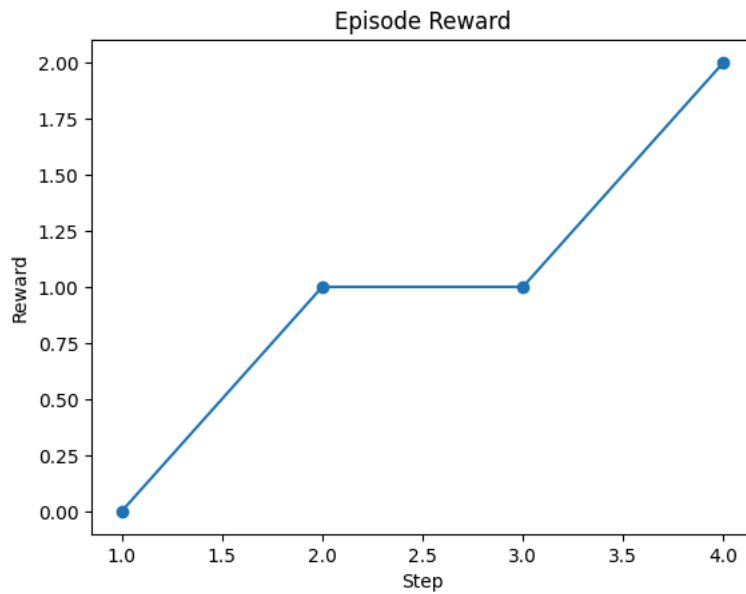
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("Episode Summary")
plt.show()

# Plot rewards
plt.plot(steps, rewards, marker="o")
plt.xlabel("Step")
plt.ylabel("Reward")
plt.title("Episode Reward")
plt.show()
```

Episode Summary

Step	State	Reward
1	A	0
2	B	1
3	B	1
4	Goal	2



```
import pandas as pd
import matplotlib.pyplot as plt

# State values
data = pd.DataFrame({
    "State": ["A", "B", "Goal"],
    "Value": [2.71, 1.90, 0]
})

# Summary table
fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

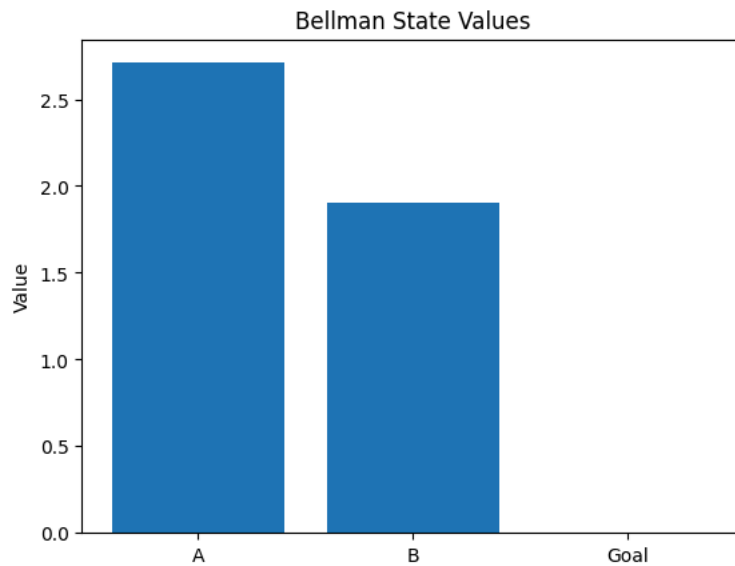
plt.title("Bellman Value Summary")
plt.show()

# Plot
states = ["A", "B", "Goal"]
values = [2.71, 1.90, 0]

plt.bar(states, values)
plt.ylabel("Value")
plt.title("Bellman State Values")
plt.show()
```

Bellman Value Summary

State	Value
A	2.71
B	1.9
Goal	0.0



```
import pandas as pd
import matplotlib.pyplot as plt

# Q-values
data = pd.DataFrame({
    "Action": ["Left", "Right"],
    "Q_Value": [0.5, 2.8]
})

# Summary table
fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

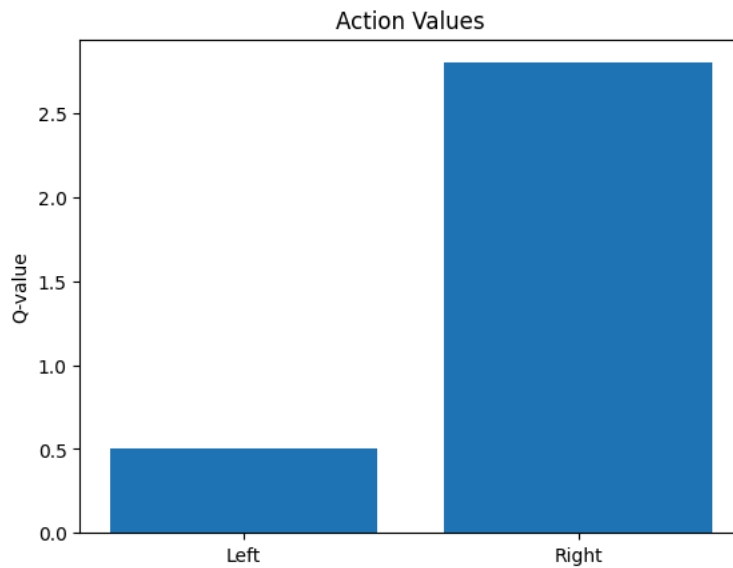
plt.title("Q-Value Summary")
plt.show()

# Plot
actions = ["Left", "Right"]
q_values = [0.5, 2.8]

plt.bar(actions, q_values)
plt.ylabel("Q-value")
plt.title("Action Values")
plt.show()
```

Q-Value Summary

Action	Q_Value
Left	0.5
Right	2.8



```
import pandas as pd
import matplotlib.pyplot as plt

# Epsilon-Greedy data
data = pd.DataFrame({
    "Action": ["A0", "A1", "A2"],
    "Selections": [20, 45, 35]
})

# Summary table
fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

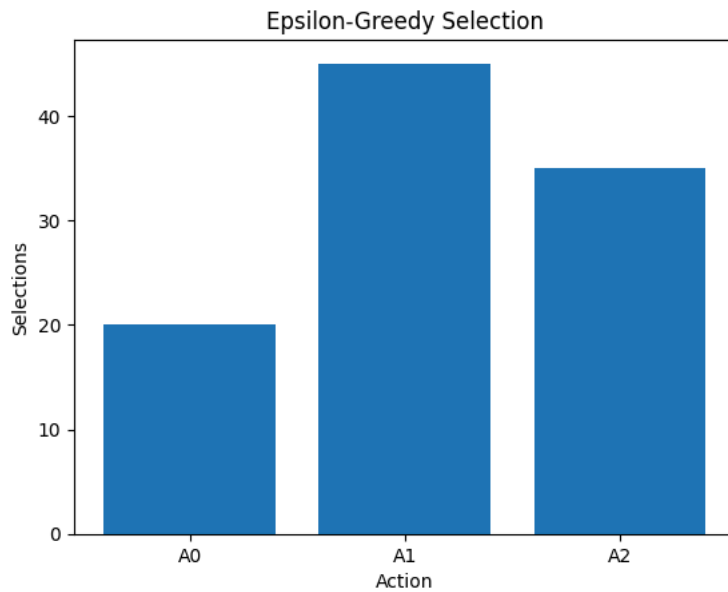
plt.title("Epsilon-Greedy Summary")
plt.show()

# Plot
actions = ["A0", "A1", "A2"]
counts = [20, 45, 35]

plt.bar(actions, counts)
plt.xlabel("Action")
plt.ylabel("Selections")
plt.title("Epsilon-Greedy Selection")
plt.show()
```

Epsilon-Greedy Summary

Action	Selections
A0	20
A1	45
A2	35



```
import pandas as pd
import matplotlib.pyplot as plt

# UCB data
data = pd.DataFrame({
    "Action": ["A1", "A2", "A3"],
    "UCB_Score": [1.8, 2.6, 2.1]
})

# Summary table
fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

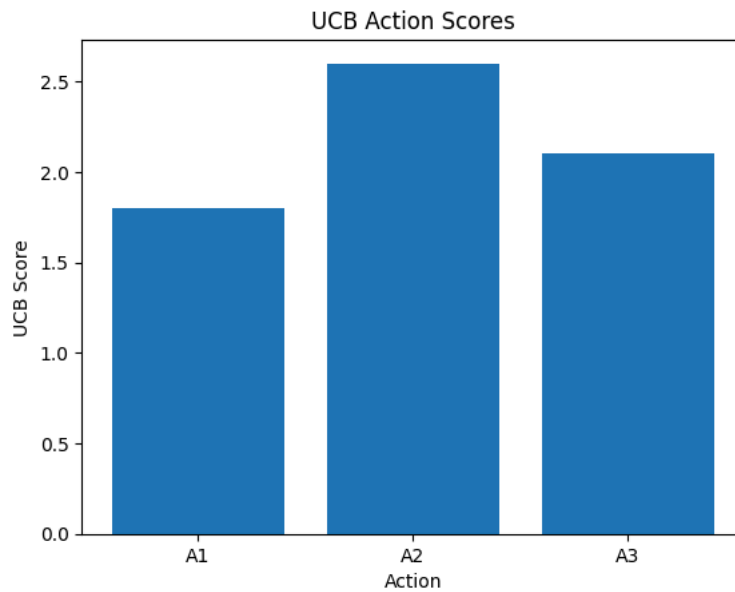
plt.title("UCB Summary")
plt.show()

# Plot
actions = ["A1", "A2", "A3"]
scores = [1.8, 2.6, 2.1]

plt.bar(actions, scores)
plt.xlabel("Action")
plt.ylabel("UCB Score")
plt.title("UCB Action Scores")
plt.show()
```


UCB Summary

Action	UCB Score
A1	1.8
A2	2.6
A3	2.1



```
import pandas as pd
import matplotlib.pyplot as plt

# Policy iteration data
data = pd.DataFrame({
    "Iteration": [1, 2, 3, 4, 5],
    "Value": [0.5, 1.0, 1.8, 2.5, 3.0]
})

# Summary table
fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

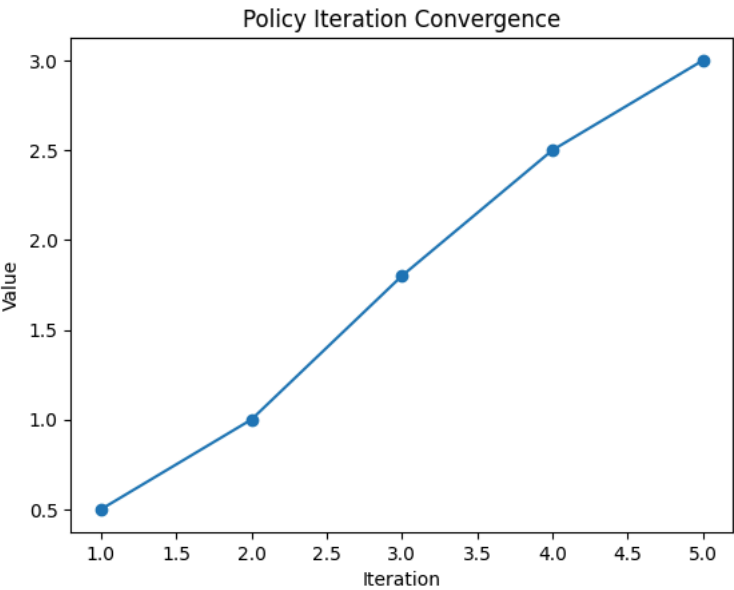
plt.title("Policy Iteration Summary")
plt.show()

# Plot
iterations = [1, 2, 3, 4, 5]
values = [0.5, 1.0, 1.8, 2.5, 3.0]

plt.plot(iterations, values, marker="o")
plt.xlabel("Iteration")
plt.ylabel("Value")
plt.title("Policy Iteration Convergence")
plt.show()
```

Policy Iteration Summary

Iteration	Value
1.0	0.5
2.0	1.0
3.0	1.8
4.0	2.5
5.0	3.0



```
import pandas as pd
import matplotlib.pyplot as plt

# Value iteration data
data = pd.DataFrame({
    "Iteration": [1, 2, 3, 4, 5],
    "Value": [0.4, 1.2, 2.0, 2.7, 3.1]
})

# Summary table
fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("Value Iteration Summary")
plt.show()

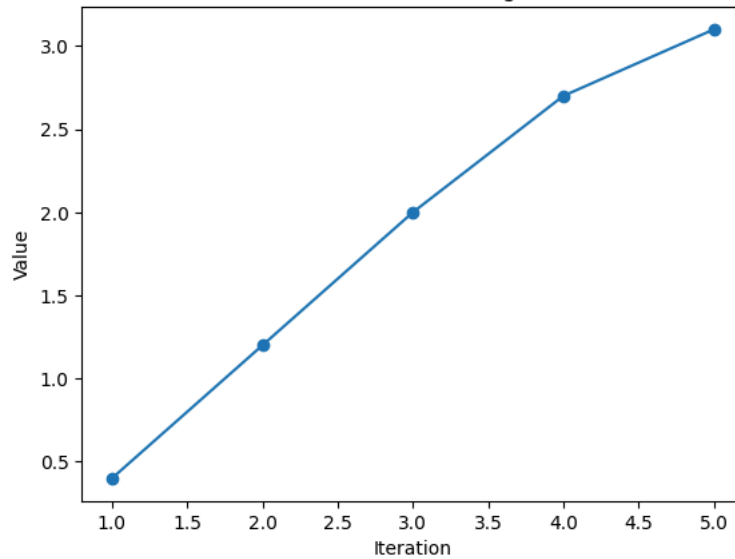
# Plot
iterations = [1, 2, 3, 4, 5]
values = [0.4, 1.2, 2.0, 2.7, 3.1]

plt.plot(iterations, values, marker="o")
plt.xlabel("Iteration")
plt.ylabel("Value")
plt.title("Value Iteration Convergence")
plt.show()
```

Value Iteration Summary

Iteration	Value
1.0	0.4
2.0	1.2
3.0	2.0
4.0	2.7
5.0	3.1

Value Iteration Convergence



```
import pandas as pd
import matplotlib.pyplot as plt

# Monte Carlo data
episodes = [1, 2, 3, 4, 5]
returns = [1, 1.8, 2.1, 2.6, 3.0]

# Create summary table
data = pd.DataFrame({
    "Episode": episodes,
    "Return": returns
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

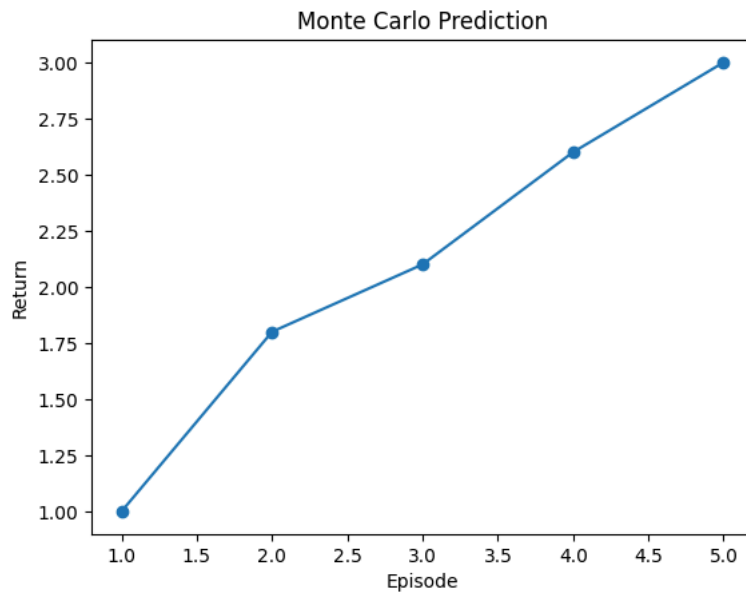
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("Monte Carlo Summary")
plt.show()

# Plot
plt.plot(episodes, returns, marker="o")
plt.xlabel("Episode")
plt.ylabel("Return")
plt.title("Monte Carlo Prediction")
plt.show()
```

Monte Carlo Summary

Episode	Return
1.0	1.0
2.0	1.8
3.0	2.1
4.0	2.6
5.0	3.0



```
import pandas as pd
import matplotlib.pyplot as plt

# Monte Carlo Control data
actions = ["Left", "Right"]
q_values = [0.8, 2.5]

# Create summary table
data = pd.DataFrame({
    "Action": actions,
    "Q_Value": q_values
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

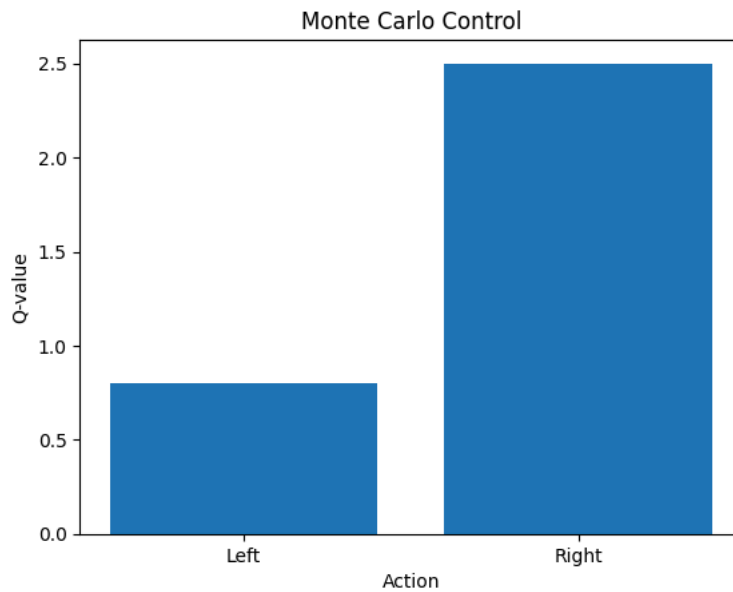
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("Monte Carlo Control Summary")
plt.show()

# Plot
plt.bar(actions, q_values)
plt.xlabel("Action")
plt.ylabel("Q-value")
plt.title("Monte Carlo Control")
plt.show()
```

Monte Carlo Control Summary

Action	Q_Value
Left	0.8
Right	2.5



```
import pandas as pd
import matplotlib.pyplot as plt

# TD(0) data
episodes = [1, 2, 3, 4, 5]
values = [0.5, 0.9, 1.3, 1.7, 2.0]

# Create summary table
data = pd.DataFrame({
    "Episode": episodes,
    "Value": values
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

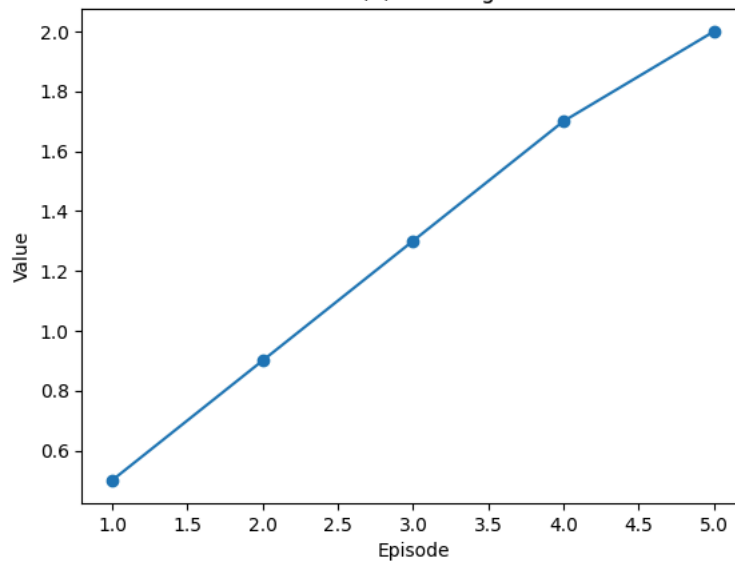
plt.title("TD(0) Summary")
plt.show()

# Plot
plt.plot(episodes, values, marker="o")
plt.xlabel("Episode")
plt.ylabel("Value")
plt.title("TD(0) Learning")
plt.show()
```

TD(0) Summary

Episode	Value
1.0	0.5
2.0	0.9
3.0	1.3
4.0	1.7
5.0	2.0

TD(0) Learning



```
import pandas as pd
import matplotlib.pyplot as plt

# SARSA data
actions = ["Left", "Right"]
q_values = [1.0, 2.4]

# Create summary table
data = pd.DataFrame({
    "Action": actions,
    "Q_Value": q_values
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

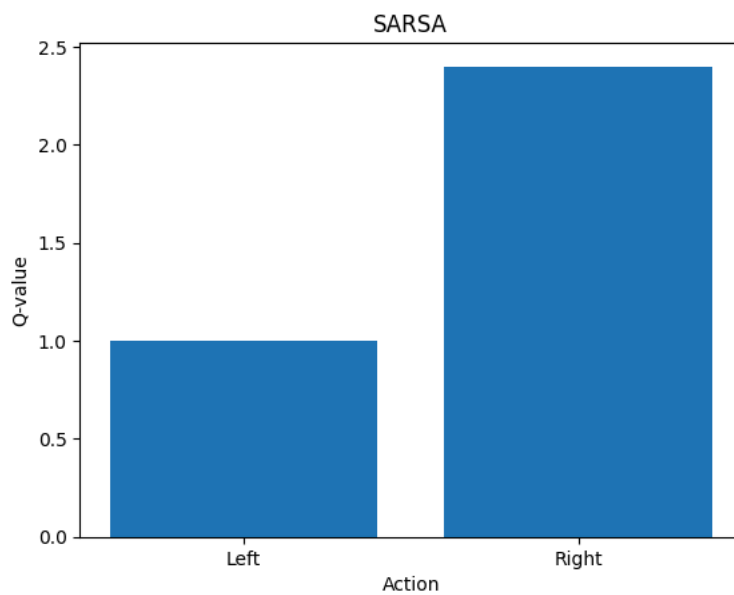
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("SARSA Summary")
plt.show()

# Plot
plt.bar(actions, q_values)
plt.xlabel("Action")
plt.ylabel("Q-value")
plt.title("SARSA")
plt.show()
```

SARSA Summary

Action	Q_Value
Left	1.0
Right	2.4



```
import pandas as pd
import matplotlib.pyplot as plt

# Q-Learning data
actions = ["Left", "Right"]
q_values = [1.2, 2.8]

# Create summary table
data = pd.DataFrame({
    "Action": actions,
    "Q_Value": q_values
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

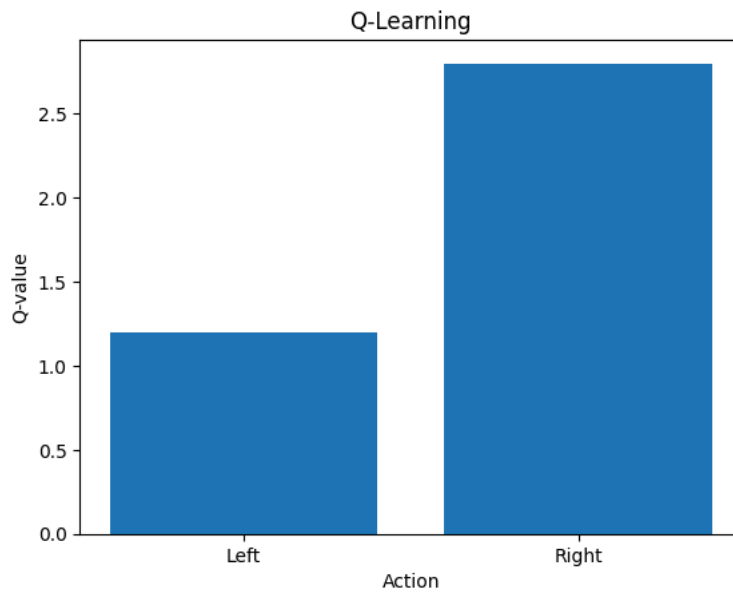
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("Q-Learning Summary")
plt.show()

# Plot
plt.bar(actions, q_values)
plt.xlabel("Action")
plt.ylabel("Q-value")
plt.title("Q-Learning")
plt.show()
```

Q-Learning Summary

Action	Q_Value
Left	1.2
Right	2.8



```
import pandas as pd
import matplotlib.pyplot as plt

# Comparison data
data = pd.DataFrame({
    "Method": ["SARSA", "Q-Learning"],
    "Final_Reward": [38, 46]
})

# Summary table
fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

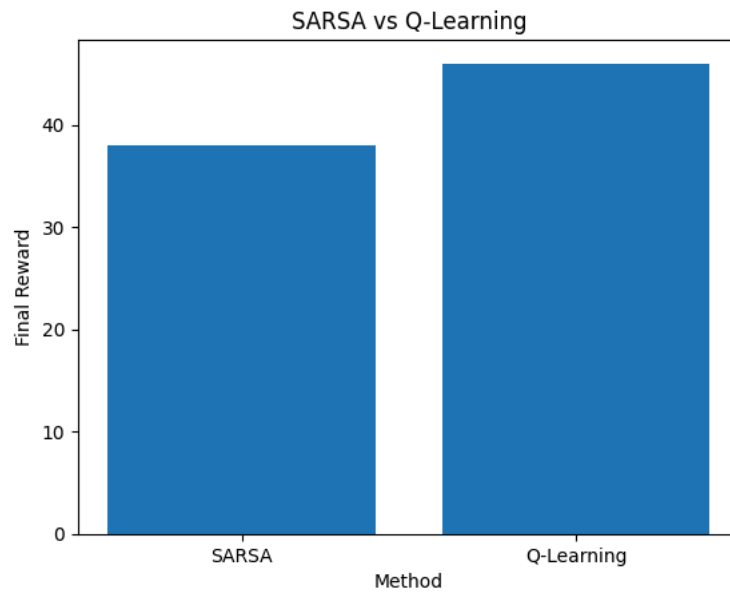
plt.title("Method Comparison")
plt.show()

# Plot
methods = ["SARSA", "Q-Learning"]
rewards = [38, 46]

plt.bar(methods, rewards)
plt.xlabel("Method")
plt.ylabel("Final Reward")
plt.title("SARSA vs Q-Learning")
plt.show()
```


Method Comparison

Method	Final_Reward
SARSA	38
Q-Learning	46



```
import pandas as pd
import matplotlib.pyplot as plt

# DQN data
episodes = [1, 2, 3, 4, 5]
rewards = [5, 10, 18, 25, 32]

# Create summary table
data = pd.DataFrame({
    "Episode": episodes,
    "Reward": rewards
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

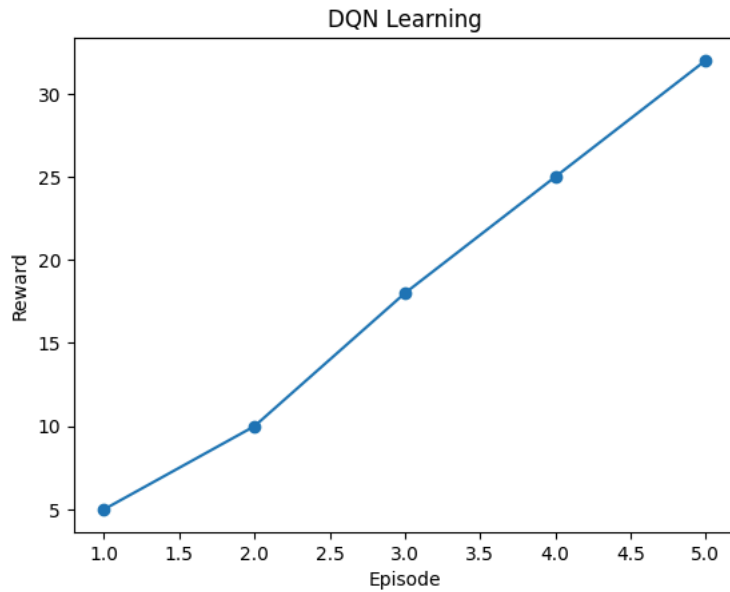
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("DQN Training Summary")
plt.show()

# Plot
plt.plot(episodes, rewards, marker="o")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("DQN Learning")
plt.show()
```

DQN Training Summary

Episode	Reward
1	5
2	10
3	18
4	25
5	32



```
import pandas as pd
import matplotlib.pyplot as plt

# Double DQN data
data = pd.DataFrame({
    "Network": ["Online Q", "Target Q"],
    "Value": [3.0, 2.5]
})

# Summary table
fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

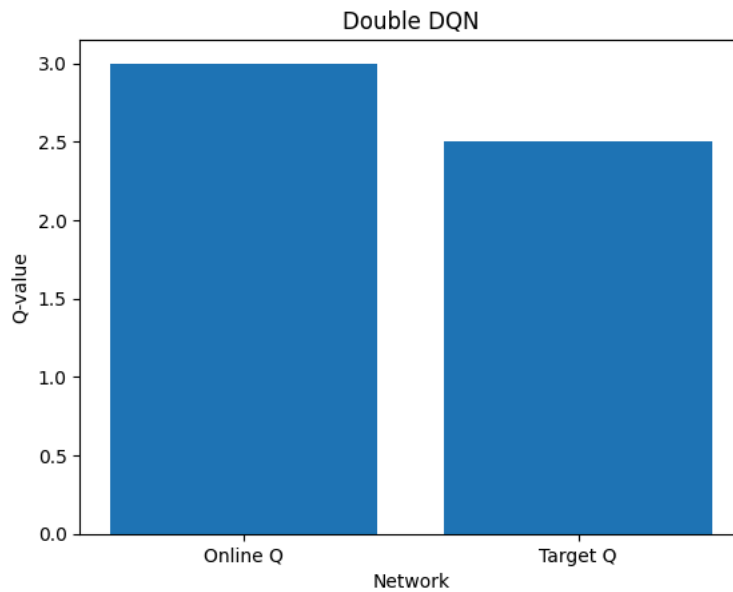
plt.title("Double DQN Summary")
plt.show()

# Plot
networks = ["Online Q", "Target Q"]
values = [3.0, 2.5]

plt.bar(networks, values)
plt.xlabel("Network")
plt.ylabel("Q-value")
plt.title("Double DQN")
plt.show()
```

Double DQN Summary

Network	Value
Online Q	3.0
Target Q	2.5



```
import pandas as pd
import matplotlib.pyplot as plt

# Dueling DQN data
data = pd.DataFrame({
    "Action": ["A1", "A2"],
    "Q_Value": [1.75, 2.25]
})

# Summary table
fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

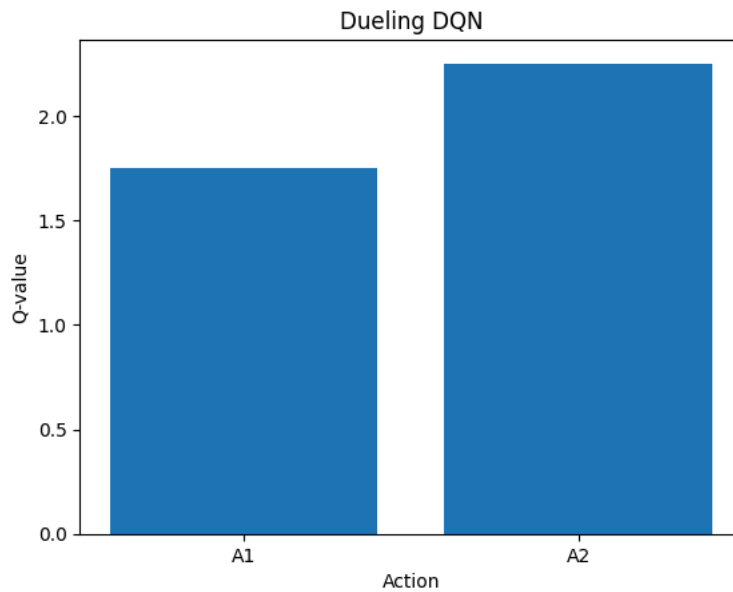
plt.title("Dueling DQN Summary")
plt.show()

# Plot
actions = ["A1", "A2"]
q_values = [1.75, 2.25]

plt.bar(actions, q_values)
plt.xlabel("Action")
plt.ylabel("Q-value")
plt.title("Dueling DQN")
plt.show()
```

Dueling DQN Summary

Action	Q_Value
A1	1.75
A2	2.25



```
import pandas as pd
import matplotlib.pyplot as plt

# Comparison data
data = pd.DataFrame({
    "Model": ["DQN", "DDQN", "Dueling DQN"],
    "Final_Reward": [25, 28, 30]
})

# Summary table
fig, ax = plt.subplots(figsize=(7, 2.5))
ax.axis("off")

ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

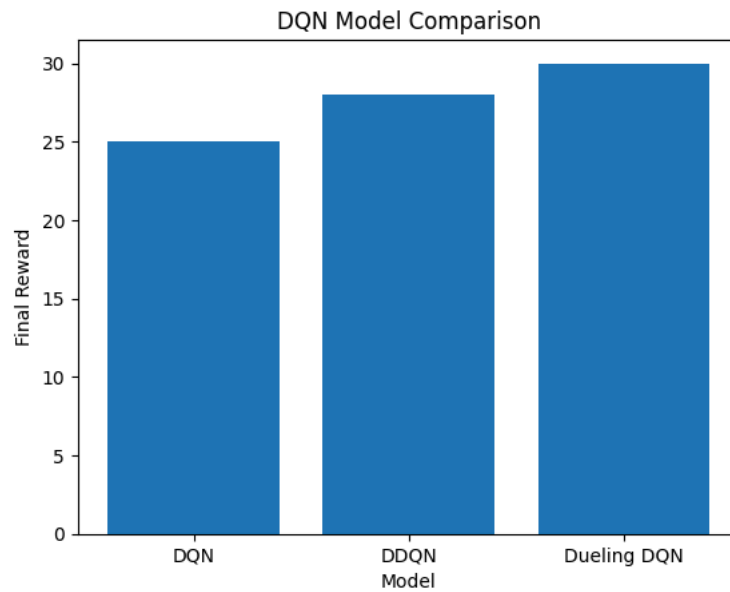
plt.title("DQN Comparison")
plt.show()

# Plot
models = ["DQN", "DDQN", "Dueling DQN"]
rewards = [25, 28, 30]

plt.bar(models, rewards)
plt.xlabel("Model")
plt.ylabel("Final Reward")
plt.title("DQN Model Comparison")
plt.show()
```

DQN Comparison

Model	Final Reward
DQN	25
DDQN	28
Dueling DQN	30



```
import pandas as pd
import matplotlib.pyplot as plt

# REINFORCE data
episodes = [1, 2, 3, 4, 5]
probability = [0.5, 0.55, 0.62, 0.7, 0.78]

# Create summary table
data = pd.DataFrame({
    "Episode": episodes,
    "Policy Probability": probability
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

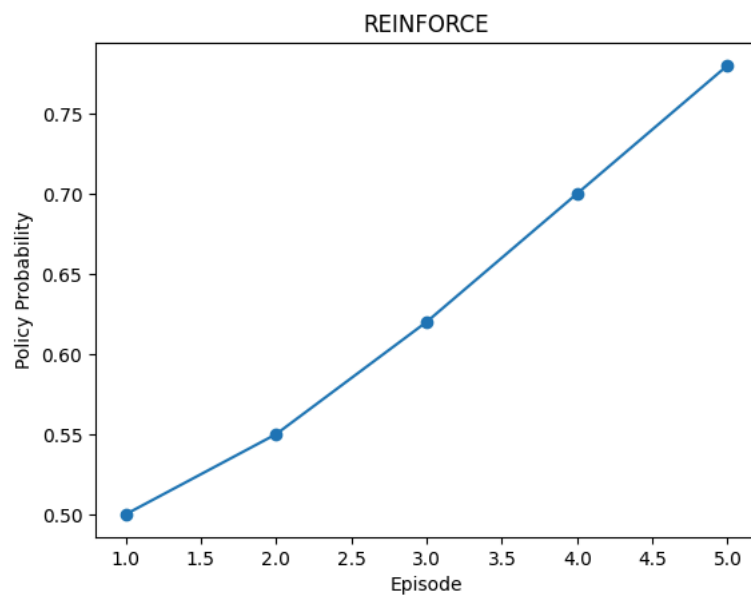
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("REINFORCE - Summary")
plt.show()

# Plot
plt.plot(episodes, probability, marker="o")
plt.xlabel("Episode")
plt.ylabel("Policy Probability")
plt.title("REINFORCE")
plt.show()
```

REINFORCE - Summary

Episode	Policy Probability
1.0	0.5
2.0	0.55
3.0	0.62
4.0	0.7
5.0	0.78



```
import pandas as pd
import matplotlib.pyplot as plt

# REINFORCE with Baseline data
episodes = [1, 2, 3, 4, 5]
advantage = [0.5, 1, 1.5, 2, 2.5]

# Create summary table
data = pd.DataFrame({
    "Episode": episodes,
    "Advantage": advantage
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

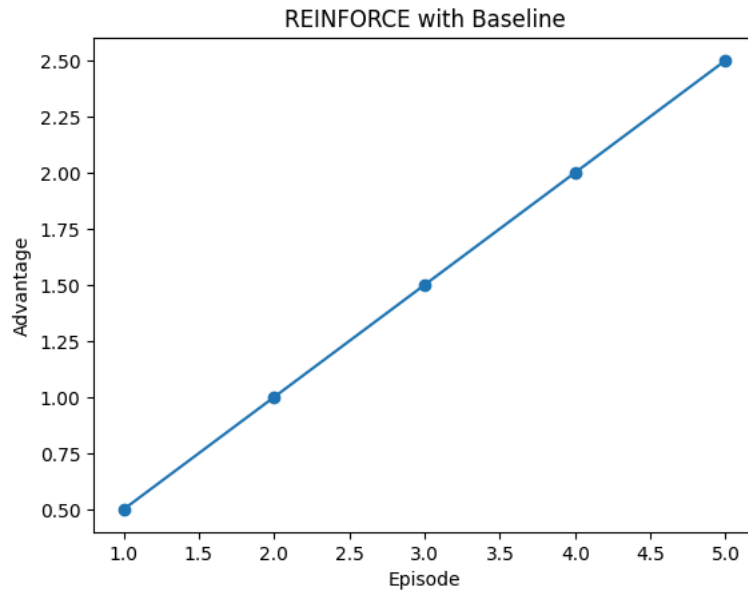
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("REINFORCE with Baseline - Summary")
plt.show()

# Plot
plt.plot(episodes, advantage, marker="o")
plt.xlabel("Episode")
plt.ylabel("Advantage")
plt.title("REINFORCE with Baseline")
plt.show()
```

REINFORCE with Baseline - Summary

Episode	Advantage
1.0	0.5
2.0	1.0
3.0	1.5
4.0	2.0
5.0	2.5



```
import pandas as pd
import matplotlib.pyplot as plt

# Actor-Critic data
steps = [1, 2, 3, 4, 5]
critic_values = [0.2, 0.5, 0.8, 1.1, 1.4]

# Create summary table
data = pd.DataFrame({
    "Step": steps,
    "Critic Value": critic_values
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

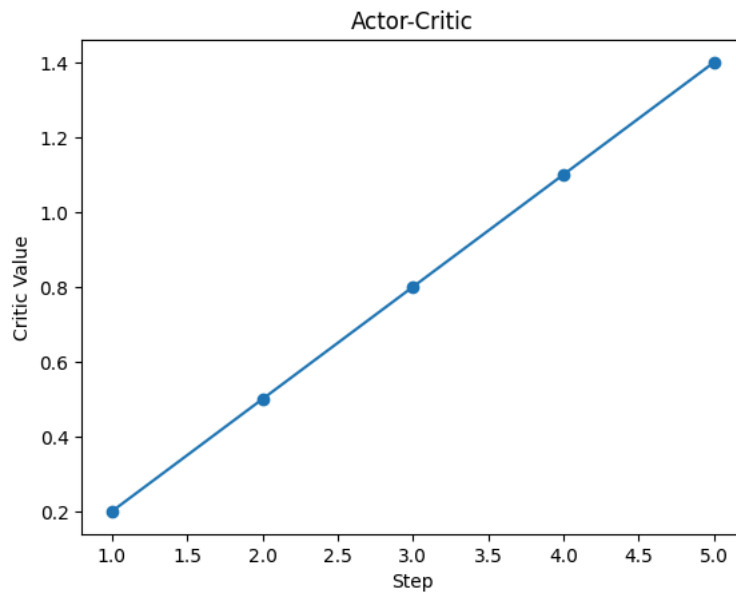
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("Actor-Critic - Summary")
plt.show()

# Plot
plt.plot(steps, critic_values, marker="o")
plt.xlabel("Step")
plt.ylabel("Critic Value")
plt.title("Actor-Critic")
plt.show()
```

Actor-Critic - Summary

Step	Critic Value
1.0	0.2
2.0	0.5
3.0	0.8
4.0	1.1
5.0	1.4



```
import pandas as pd
import matplotlib.pyplot as plt

# A2C data
episodes = [1, 2, 3, 4, 5]
advantage = [0.5, 1, 1.6, 2.1, 2.4]

# Create summary table
data = pd.DataFrame({
    "Episode": episodes,
    "Advantage": advantage
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

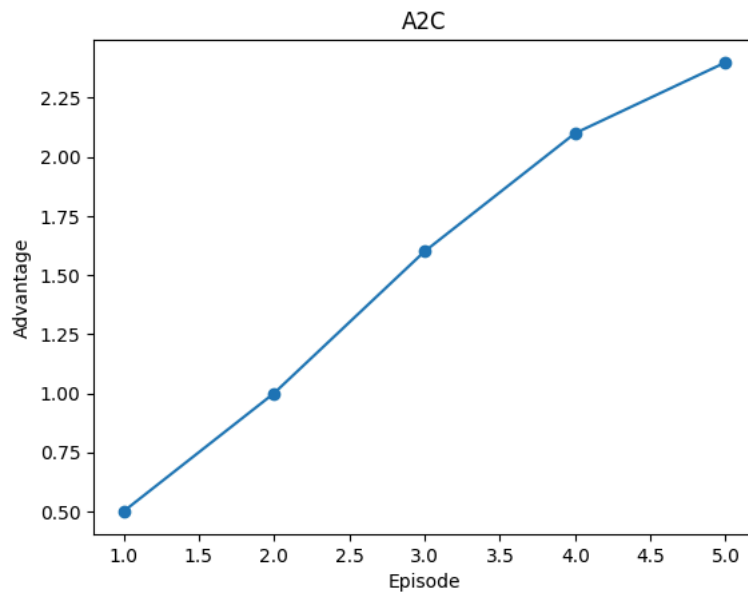
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("A2C - Summary")
plt.show()

# Plot
plt.plot(episodes, advantage, marker="o")
plt.xlabel("Episode")
plt.ylabel("Advantage")
plt.title("A2C")
plt.show()
```


A2C - Summary

Episode	Advantage
1.0	0.5
2.0	1.0
3.0	1.6
4.0	2.1
5.0	2.4



```
import pandas as pd
import matplotlib.pyplot as plt

# A3C data
workers = [1, 2, 3, 4]
results = [1, 4, 9, 16]

# Create summary table
data = pd.DataFrame({
    "Worker": workers,
    "Result": results
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

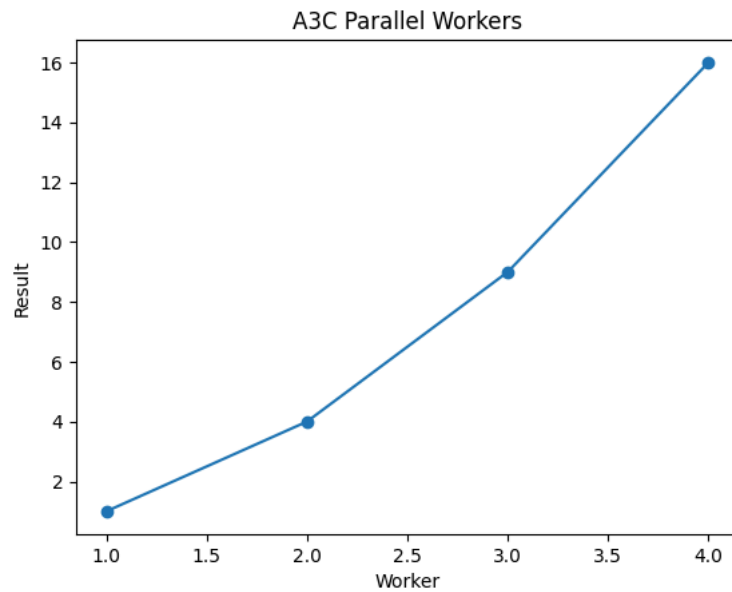
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("A3C Parallel Workers - Summary")
plt.show()

# Plot
plt.plot(workers, results, marker="o")
plt.xlabel("Worker")
plt.ylabel("Result")
plt.title("A3C Parallel Workers")
plt.show()
```

A3C Parallel Workers - Summary

Worker	Result
1	1
2	4
3	9
4	16



```
import pandas as pd
import matplotlib.pyplot as plt

# A2C vs A3C data
episodes = [1, 2, 3, 4]
a2c_values = [10, 20, 30, 40]

# Create summary table
data = pd.DataFrame({
    "Episode": episodes,
    "A2C": a2c_values
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

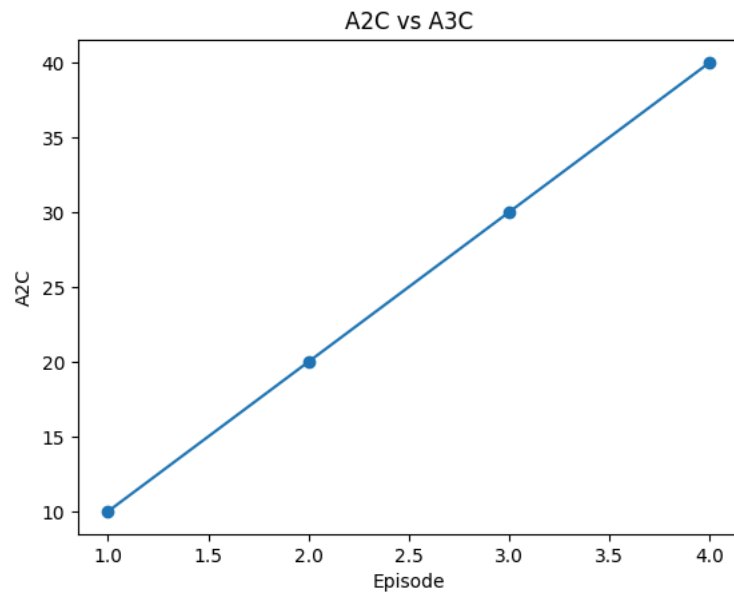
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("A2C vs A3C - Summary")
plt.show()

# Plot
plt.plot(episodes, a2c_values, marker="o")
plt.xlabel("Episode")
plt.ylabel("A2C")
plt.title("A2C vs A3C")
plt.show()
```

A2C vs A3C - Summary

Episode	A2C
1	10
2	20
3	30
4	40



```
import pandas as pd
import matplotlib.pyplot as plt

# PPO data
updates = [1, 2, 3, 4, 5]
objective = [1, 1.4, 1.7, 1.9, 2]

# Create summary table
data = pd.DataFrame({
    "Update": updates,
    "Objective": objective
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

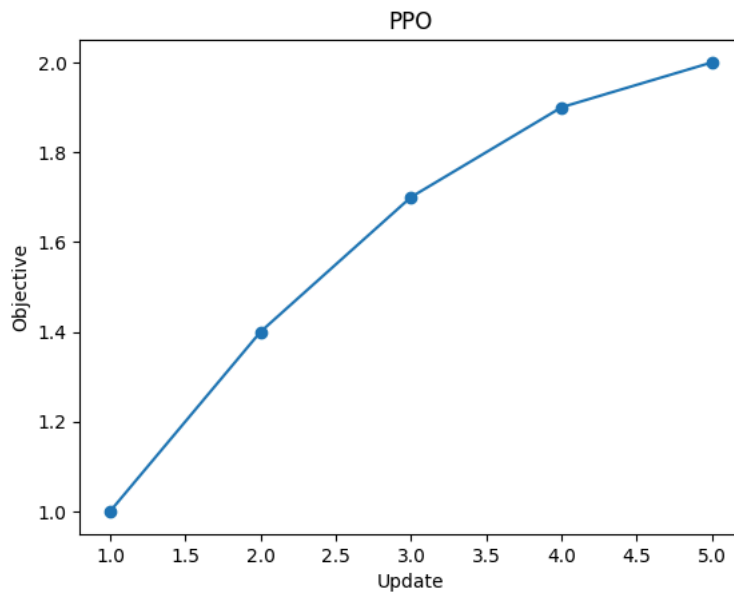
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("PPO - Summary")
plt.show()

# Plot
plt.plot(updates, objective, marker="o")
plt.xlabel("Update")
plt.ylabel("Objective")
plt.title("PPO")
plt.show()
```

PPO - Summary

Update	Objective
1.0	1.0
2.0	1.4
3.0	1.7
4.0	1.9
5.0	2.0



```
import pandas as pd
import matplotlib.pyplot as plt

# Continuous PPO data
actions = [1, 2, 3]
values = [0.7, 0.73, 1.0]

# Create summary table
data = pd.DataFrame({
    "Action": actions,
    "Value": values
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

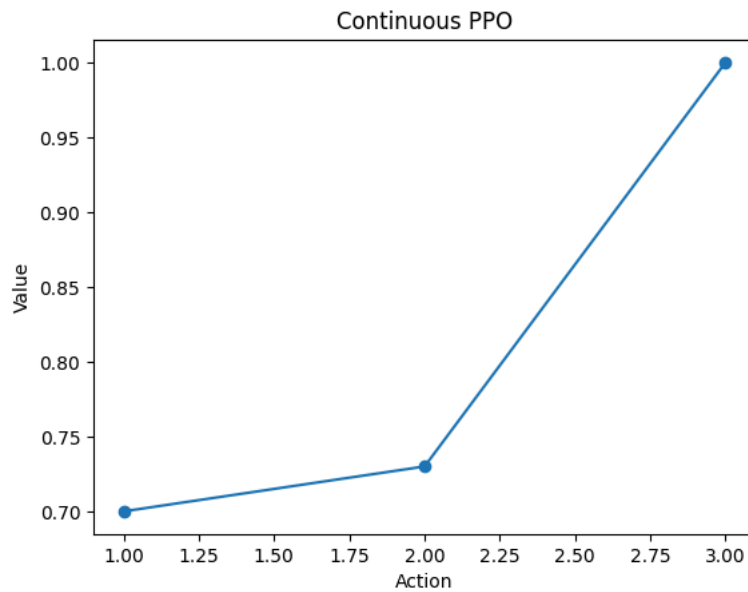
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("Continuous PPO - Summary")
plt.show()

# Plot
plt.plot(actions, values, marker="o")
plt.xlabel("Action")
plt.ylabel("Value")
plt.title("Continuous PPO")
plt.show()
```

Continuous PPO - Summary

Action	Value
1.0	0.7
2.0	0.73
3.0	1.0



```
import pandas as pd
import matplotlib.pyplot as plt

# Environment model data
states = [1, 2, 3, 4]
next_states = [1, 2, 3, 4]

# Create summary table
data = pd.DataFrame({
    "State": states,
    "Next State": next_states
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

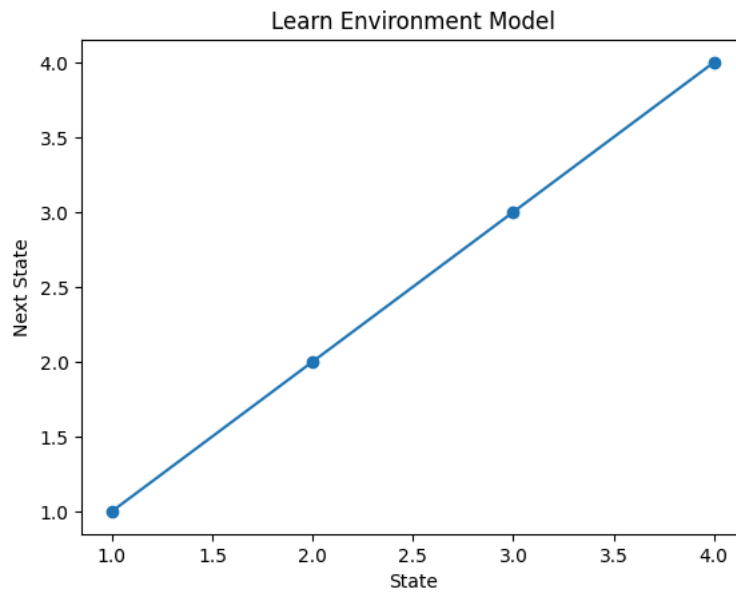
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("Learn Environment Model - Summary")
plt.show()

# Plot
plt.plot(states, next_states, marker="o")
plt.xlabel("State")
plt.ylabel("Next State")
plt.title("Learn Environment Model")
plt.show()
```

Learn Environment Model - Summary

State	Next State
1	1
2	2
3	3
4	4



```
import pandas as pd
import matplotlib.pyplot as plt

# One-Step Planning data
states = [1, 2, 3, 4]
rewards = [0, 0, 1, 5]

# Create summary table
data = pd.DataFrame({
    "State": states,
    "Reward": rewards
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

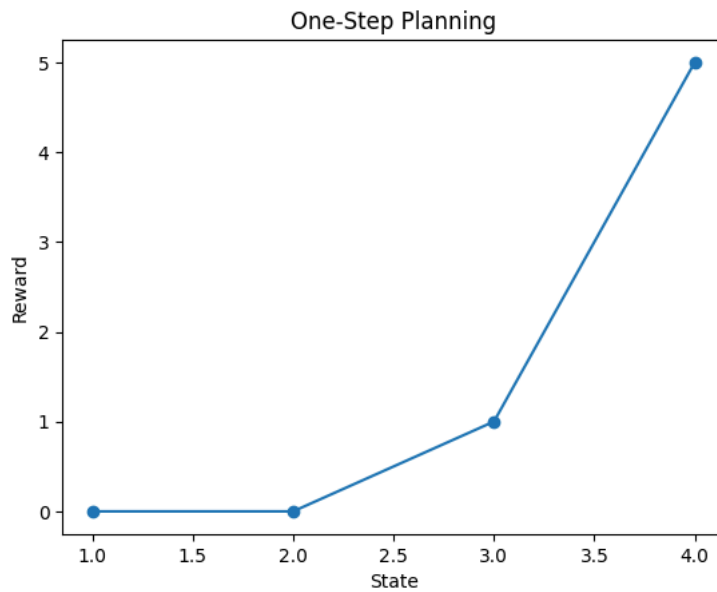
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("One-Step Planning - Summary")
plt.show()

# Plot
plt.plot(states, rewards, marker="o")
plt.xlabel("State")
plt.ylabel("Reward")
plt.title("One-Step Planning")
plt.show()
```

One-Step Planning - Summary

State	Reward
1	0
2	0
3	1
4	5



```
import pandas as pd
import matplotlib.pyplot as plt

# Sampling-Based Planning data
actions = [1, 2]
scores = [9.5, 14.2]

# Create summary table
data = pd.DataFrame({
    "Action": actions,
    "Score": scores
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

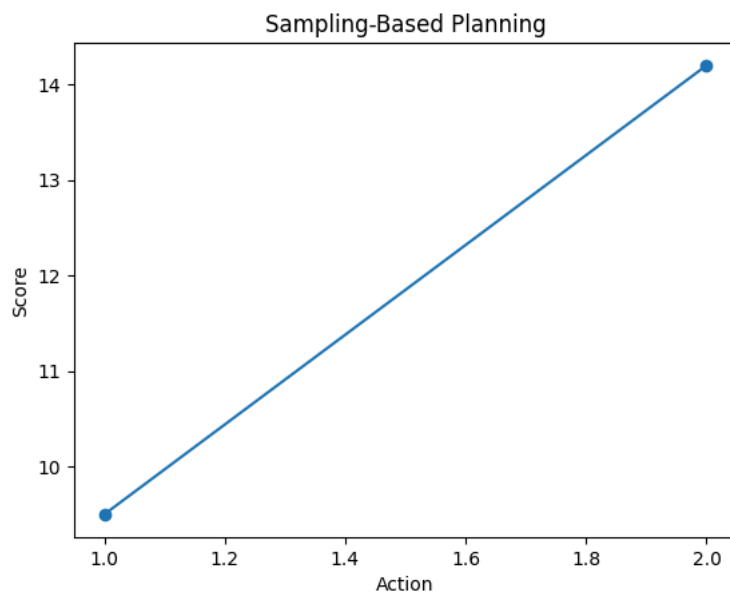
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("Sampling-Based Planning - Summary")
plt.show()

# Plot
plt.plot(actions, scores, marker="o")
plt.xlabel("Action")
plt.ylabel("Score")
plt.title("Sampling-Based Planning")
plt.show()
```

Sampling-Based Planning - Summary

Action	Score
1.0	9.5
2.0	14.2



```
import pandas as pd
import matplotlib.pyplot as plt

# Best Sampled Sequence data
actions = [1, 2]
returns = [4, 8]

# Create summary table
data = pd.DataFrame({
    "Action": actions,
    "Return": returns
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

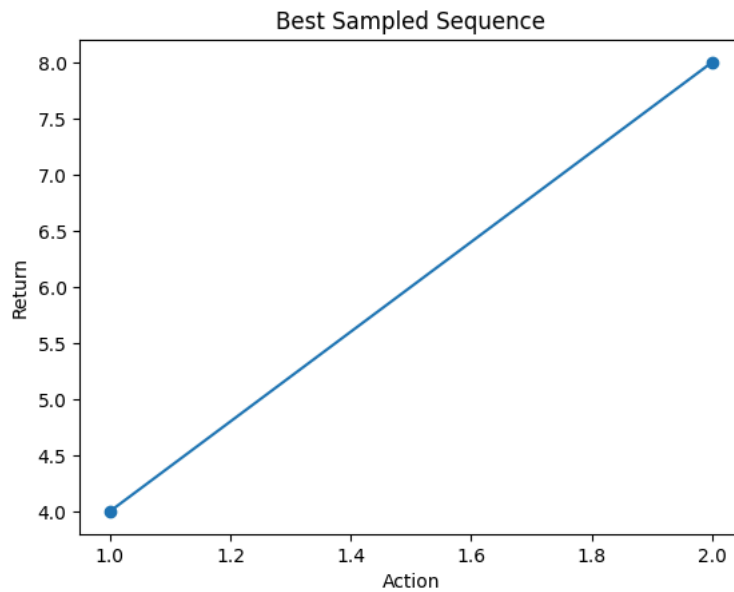
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("Best Sampled Sequence - Summary")
plt.show()

# Plot
plt.plot(actions, returns, marker="o")
plt.xlabel("Action")
plt.ylabel("Return")
plt.title("Best Sampled Sequence")
plt.show()
```


Best Sampled Sequence - Summary

Action	Return
1	4
2	8



```
import pandas as pd
import matplotlib.pyplot as plt

# Model-Based Policy Optimization data
episodes = [1, 2, 3, 4, 5]
rewards = [3, 5, 7, 8, 10]

# Create summary table
data = pd.DataFrame({
    "Episode": episodes,
    "Reward": rewards
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

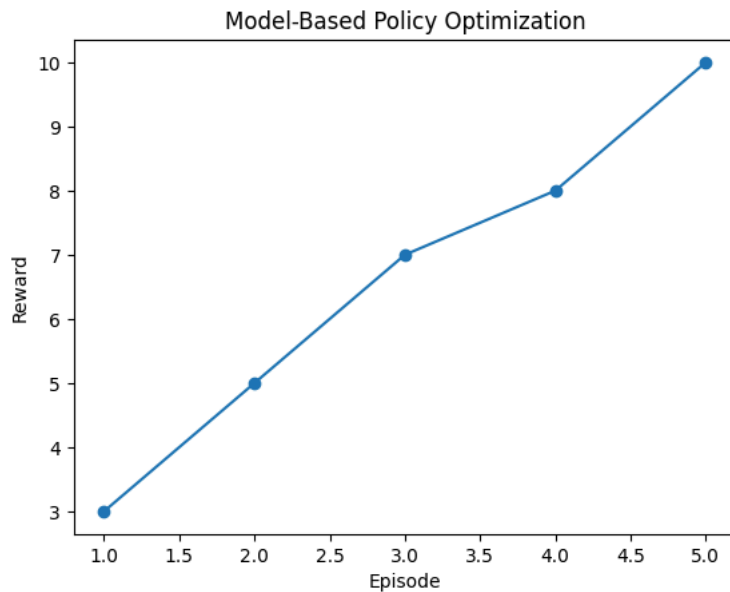
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("Model-Based Policy Optimization - Summary")
plt.show()

# Plot
plt.plot(episodes, rewards, marker="o")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("Model-Based Policy Optimization")
plt.show()
```

Model-Based Policy Optimization - Summary

Episode	Reward
1	3
2	5
3	7
4	8
5	10



```
import pandas as pd
import matplotlib.pyplot as plt

# Model-Based vs Model-Free data
methods = [1, 2]
samples = [100, 250]

# Create summary table
data = pd.DataFrame({
    "Method": methods,
    "Samples": samples
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

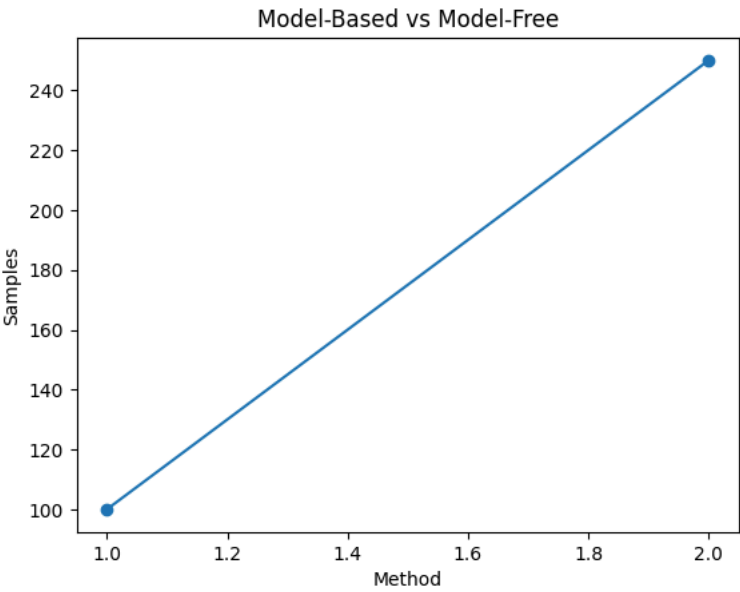
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("Model-Based vs Model-Free - Summary")
plt.show()

# Plot
plt.plot(methods, samples, marker="o")
plt.xlabel("Method")
plt.ylabel("Samples")
plt.title("Model-Based vs Model-Free")
plt.show()
```

Model-Based vs Model-Free - Summary

Method	Samples
1	100
2	250



```
import pandas as pd
import matplotlib.pyplot as plt

# Value-Equivalence Prediction data
states = [1, 2, 3, 4]
errors = [2, 1.5, 1, 0.5]

# Create summary table
data = pd.DataFrame({
    "State": states,
    "Error": errors
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

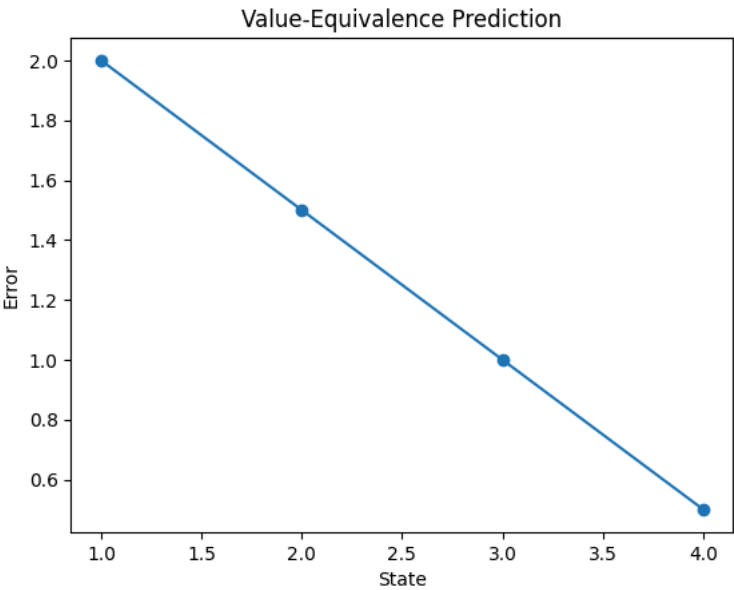
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("Value-Equivalence Prediction - Summary")
plt.show()

# Plot
plt.plot(states, errors, marker="o")
plt.xlabel("State")
plt.ylabel("Error")
plt.title("Value-Equivalence Prediction")
plt.show()
```

Value-Equivalence Prediction - Summary

State	Error
1.0	2.0
2.0	1.5
3.0	1.0
4.0	0.5



```
import pandas as pd
import matplotlib.pyplot as plt

# Actual vs Predicted data
states = [1, 2, 3, 4]
actual = [5, 7, 9, 10]

# Create summary table
data = pd.DataFrame({
    "State": states,
    "Actual": actual
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

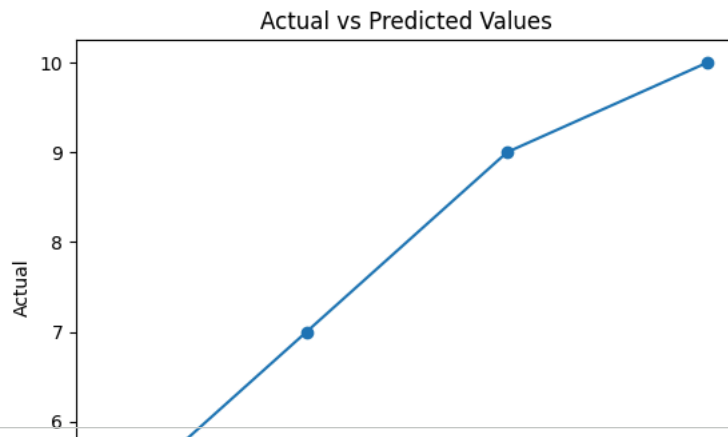
ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("Actual vs Predicted Values - Summary")
plt.show()

# Plot
plt.plot(states, actual, marker="o")
plt.xlabel("State")
plt.ylabel("Actual")
plt.title("Actual vs Predicted Values")
plt.show()
```

Actual vs Predicted Values - Summary

State	Actual
1	5
2	7
3	9
4	10



```
import pandas as pd
import matplotlib.pyplot as plt

# MAXQ data
tasks = [1, 2, 3]
rewards = [1, 2, 4]

# Create summary table
data = pd.DataFrame({
    "Task": tasks,
    "Reward": rewards
})

fig, ax = plt.subplots(figsize=(6, 2.5))
ax.axis("off")

ax.table(
    cellText=data.values,
    colLabels=data.columns,
    loc="center"
)

plt.title("MAXQ-Style Hierarchical Task - Summary")
plt.show()
```